Next.js Deployment on cPanel Shared Hosting

Project: nextjs-multivendor-ecommerce · Live at: nextjs-ecommerce.rioncreates.com · Deployed: 18 Oct 2026

1. Overview

This document records how the Next.js multivendor e-commerce project was deployed to a cPanel shared hosting account using the **Setup Node.js App** feature (standalone build). It also lists the mistakes made along the way and how each one was fixed, so future deployments are faster.

Item	Value
Local project path	`D:\NextJS\nextjs-multivendor-ecommerce`
Local OS / Node version	Windows, Node v26.5.0
Hosting	cPanel shared hosting (cPanel 138.0.15)
Server Node version used	24.21.0 (highest available)
Subdomain	`nextjs-ecommerce.rioncreates.com`
Application root (server)	`nextjs-ecommerce-app` (in the home directory, outside `public_html`)
Startup file	`server.js`
Deployment method	Next.js `output: 'standalone'` build, uploaded as a zip

2. Choosing the deployment method

Two options exist for Next.js on cPanel:

- **Static export** (`output: 'export'`): works on any host but has no server-side rendering, API routes or middleware.
- **Node.js app** (`output: 'standalone'`): supports all Next.js features but requires the **Setup Node.js App** tool in cPanel.

The cPanel account was checked for **Setup Node.js App**, and the page was present (with "No applications found"). Because an e-commerce site needs API routes and a database, the **Node.js app method** was chosen.

3. Step-by-step deployment

Step 1: Enable standalone output

In `next.config.js` (or `next.config.mjs`):

```
const nextConfig = {
  output: 'standalone',
};
module.exports = nextConfig;
```

Step 2: Build locally

The build was done on the local PC, not on the server, because shared hosting often does not have enough RAM for `next build`.

```
npm install
npm run build
```

This creates `.next/standalone`, which contains `server.js`, `package.json`, `node_modules`, `.next` and (in this project) `.env`.

Step 3: Assemble the `deploy` folder

The standalone output is **not complete on its own**. It does not include the `public` folder or `.next/static`, so those must be added. This was done **manually in File Explorer** (see section 5 for the manual steps and the equivalent PowerShell commands).

Key rule: every file and folder inside `.next/standalone` must be copied into the `deploy` folder. The standalone folder holds the runnable server (`server.js`, `node_modules`, the compiled `.next` output and `.env`), so it is the base of the deployment. Copy its *contents* into `deploy`, not the `standalone` folder itself. Then add the two things standalone leaves out: `.next/static` and `public`.

1. In the project root (`D:\NextJS\nextjs-multivendor-ecommerce`), create a new empty folder named `deploy`.
2. In File Explorer, turn on **View → Show → Hidden items**, otherwise the `.next` folder and `.env` file will not be visible and will be missed.
3. Open `.next\standalone`. It contains five items: the `.next` folder, the `node_modules` folder, `.env`, `package.json` and `server.js`.
4. Press **Ctrl+A** to select all five items, copy them, and paste them directly into `deploy`.
5. Go back to the project's `.next\static` folder (this is the build's own `.next`, not the one inside `standalone`). Copy the whole `static` folder into `deploy\.next\`, so the path becomes `deploy\.next\static`.
6. Copy the project's `public` folder into `deploy`, so the path becomes `deploy\public`.
7. Check `deploy`: it must show `.next`, `node_modules`, `public`, `.env`, `package.json` and `server.js`. Open `deploy\.next` and confirm it contains both `server` and `static`.

Summary of what is copied from where:

What	Copy from	Copy to
`.next` (server files, manifests, `BUILD_ID`), `node_modules`, `.env`, `package.json`, `server.js`	`.next\standalone\` (all contents)	`deploy\`
`static` folder	`.next\static`	`deploy\.next\static`
`public` folder	`public`	`deploy\public`

Common error: copying the `standalone` folder itself gives `deploy\standalone\server.js`, which hides `server.js` one level too deep and stops the app from starting (this was Mistake 5 below). `server.js` must sit directly inside `deploy`.

The finished `deploy` folder must look like this:

```
deploy/
  .next/
    server/
    static/          <- added manually
    BUILD_ID, *.json manifests
  node_modules/
  public/            <- added manually
  .env
  package.json
  server.js
```

Step 4: Zip the contents

1. Open the `deploy` folder and press **Ctrl+A** to select everything inside it.

2. Right-click, **Add to archive**, and name it `deploy.zip`.
3. Open the zip and confirm `server.js` is visible immediately, not inside another `deploy` folder.

Step 5: Prepare the server folder

1. In cPanel **File Manager**, go to the home directory (above `public_html`).
2. Create the folder `nextjs-ecommerce-app`.
3. Turn on **Settings → Show Hidden Files (dotfiles)** so `.next` and `.env` are visible.
4. Upload `deploy.zip` into `nextjs-ecommerce-app`, right-click it, **Extract**, then delete the zip.
5. Confirm `server.js`, `package.json`, `.next`, `node_modules`, `public` and `.env` are directly inside the folder.

Step 6: Create the Node.js application

In **cPanel → Setup Node.js App → Create Application**:

Field	Value
Node.js version	24.21.0
Application mode	Production
Application root	`nextjs-ecommerce-app`
Application URL	`nextjs-ecommerce.rioncreates.com`
Application startup file	`server.js`

Click **Create**, then **Restart**. The site loaded successfully.

4. Mistakes made and how they were solved

Mistake 1: Unsure whether the host supported Node.js apps

Problem: It was unclear whether the static export or the Node.js method could be used.

Solution: The cPanel **Setup Node.js App** page was opened and checked. Since it existed, the Node.js method was usable.

Mistake 2: Worry about the Node version mismatch

Problem: The local PC ran Node v26.5.0, but the host's highest version was 24.21.0.

Solution: Node 24.21.0 was selected on the host. Next.js only needs a minimum version, and the built output is plain JavaScript, so a slightly older Node on the server is fine. Only native modules (such as `sharp`) can be an issue when building on Windows and running on Linux. If that happens, click **Run NPM Install** on the Node.js App page.

Mistake 3: Confusion about the Application root

Problem: It was unclear what to type in the **Application root** field, and the subdomain already had its own folder (`nextjs-ecommerce.rioncreates.com`).

Solution: The Application root is a folder path relative to the home directory (for example `nextjs-ecommerce-app`, no leading slash). The subdomain's folder is its public document root, so putting the app (with `.env` and source files) there risks exposing it. A separate folder outside `public_html` was used instead, and the subdomain URL was attached to it in the Node.js App form.

Mistake 4: Created the server folder but forgot to upload the files first

Problem: The folder `nextjs-ecommerce-app` existed, but the upload and extract step had been skipped.

Solution: The folder contents were checked. When a folder is empty, it only needs the zip uploaded and

extracted. When it holds wrong or partial files, delete everything inside it and upload again. Hidden files must be visible to verify the result.

Mistake 5: `server.js` was buried inside the `standalone` folder

Problem: The first `deploy.zip` contained `.next`, `public` and `standalone` side by side. `server.js` was inside `standalone`, not at the top level, so the app could not start.

Solution: The **contents** of `standalone` (not the folder itself) were copied into a fresh `deploy` folder, so that `server.js` sat at the top level.

Mistake 6: `public` and `.next/static` were missing

Problem: The standalone output does not copy `public` or `.next/static`. Without them the site loads with no CSS, JavaScript or images.

Solution: `.next/static` was copied into `deploy/.next/static`, and `public` was copied into `deploy/public`. The result was verified by opening `deploy/.next` and checking that both `server` and `static` were present.

Mistake 7: Zip contained a wrapper folder

Problem: The archive opened as `deploy.zip\deploy`, which means files were nested one level too deep inside a `deploy` folder.

Solution: Open the `deploy` folder, select everything **inside** it (Ctrl+A) and add that to the archive, then open the zip to verify that `server.js` is visible immediately.

5. Creating the `deploy` folder

The `deploy` folder in this project was created **manually in File Explorer**. The equivalent PowerShell commands are provided for faster future deployments.

Option A: Manual method (what was actually done)

1. In **File Explorer**, open `D:\NextJS\nextjs-multivendor-ecommerce`.
2. Create a new empty folder named `deploy`.
3. Turn on **View → Show → Hidden items** so the `.next` folder and `.env` file are visible.
4. Open `.next\standalone`, press **Ctrl+A**, copy, and paste everything into `deploy`. This brings in `.next`, `node_modules`, `.env`, `package.json` and `server.js`.
5. Open the project's `.next\static` folder and copy it to `deploy\.next\static` (the `.next` folder in `deploy` already exists from step 4, so paste `static` inside it).
6. Copy the project's `public` folder into `deploy\public`.
7. Verify that `deploy` shows `.next`, `node_modules`, `public`, `.env`, `package.json`, `server.js`, and that `deploy\.next` shows `server` and `static`.

Option B: PowerShell method

Open PowerShell in the project folder (the one containing `package.json`) after running `npm run build`:

```
# Remove any old deploy folder and create a fresh one
if (Test-Path deploy) { Remove-Item -Recurse -Force deploy }
New-Item -ItemType Directory deploy

# Copy the CONTENTS of standalone (includes hidden .next and .env)
Copy-Item -Recurse -Force .next\standalone\* deploy\
Copy-Item -Force .next\standalone\.env deploy\ -ErrorAction SilentlyContinue

# Add the static assets and public folder
New-Item -ItemType Directory -Force deploy\.next\static
Copy-Item -Recurse -Force .next\static\* deploy\.next\static\
Copy-Item -Recurse -Force public deploy\public
```

Optional: create the zip from the contents of `deploy` (so `server.js` is at the top level):

```
Compress-Archive -Path deploy\* -DestinationPath deploy.zip -Force
```

> Note: `Compress-Archive` may skip hidden items such as `.next` and `.env` on some PowerShell versions. If the zip is missing them, create the archive with WinRAR or 7-Zip instead (select everything inside `deploy`, then Add to archive).

6. Redeploying after code changes

1. Run `npm run build` locally.
2. Recreate the `deploy` folder (manually or with the PowerShell commands above).
3. Zip the contents of `deploy`.
4. In File Manager, open `nextjs-ecommerce-app`, upload the zip and extract it over the existing files (or delete old files first for a clean upload).
5. Delete the zip.
6. Go to **Setup Node.js App** and click **Restart**.

7. Post-deployment checklist

- Test product pages, login and register, cart, checkout, and the admin or vendor dashboard.
- Update `.env` values that point to local development (such as `NEXTAUTH_URL=http://localhost:3000` or a local database URL) to the live subdomain and production database, then restart.
- Confirm `https://nextjs-ecommerce.rioncreates.com` loads without a certificate warning. If not, run AutoSSL under **Security → SSL/TLS Status**.
- Keep a copy of `deploy.zip` and take a database backup so you can roll back.
- Watch the hosting resource usage. If the site becomes slow or crashes under traffic, consider a VPS.

8. Troubleshooting quick reference

Symptom	Likely cause	Fix
503 or "Application failed to start"	Wrong startup file, nested `server.js`, wrong Node version	Check `stderr.log`; ensure `server.js` is directly in the Application root
Site loads without CSS or images	`.next/static` or `public` missing	Copy them into `deploy` and re-upload
Error mentioning `sharp` or a native module	Built on Windows, running on Linux	Click **Run NPM Install**, then Restart, or set `images: { unoptimized: true }`
Changes not showing after upload	App not restarted	Click **Restart** in Setup Node.js App
Database connection error	Wrong host or user not attached to database	Use `localhost` as host on cPanel and add the user to the database
Cannot see `.next` or `.env`	Hidden files are off	File Manager → Settings → Show Hidden Files

9. Lessons learned

- Always build locally, then upload; do not run `next build` on shared hosting.
- The standalone folder is **not** self-contained: always add `.next/static` and `public`.
- Copy the **contents** of `standalone`, and zip the **contents** of `deploy`. Never include wrapper folders.
- Keep the app root outside `public_html`.
- Verify the folder structure at every stage (before zipping, after zipping, after extracting).